

HomeMind

端侧智能家居智能体项目报告

端侧算力约束下的场景化智能体系统

文档信息

项目名称:	HomeMind 端侧智能家居智能体
文档类型:	端侧AI智能体竞赛项目报告
团队名称:	这里填写团队名称
负责人:	这里填写姓名
指导教师:	这里填写指导教师
学院专业:	这里填写学院与专业
完成日期:	2026 年 4 月 27 日

封面主视觉占位符

项目主界面 / 系统总览图 / Logo 组合图

摘要

随着端侧 AI 算力持续提升，智能家居开始从“设备联网”进入“家庭终端智能体”阶段。HomeMind 的方案目标不是简单叠加语音、问答与控制功能，而是在赛题 4 GB 内存上限与系统常态业务 12 GB 预算内，设计一套能够长期运行在家庭终端上的端侧 Agent 主链路，使其同时具备自然语言理解、上下文记忆、偏好适配、结构化决策与设备执行能力。

在总体架构上，HomeMind 采用“端侧优先、端云协同增强”的设计思路：高频明确指令在本地完成理解、决策与执行闭环，少量模糊意图通过置信度路由进入云侧协同裁决。在理解链路上，BSR 广召回模块以规则、向量、历史记忆三路并行方式快速定位候选动作；LSR 轻量重排序模块以 5 维环境与偏好特征完成上下文感知排序；DQN 主动推荐模块则负责在合适时机给出场景级建议。在记忆设计上，系统采用“会话上下文 + 结构化偏好 + 可检索长期记忆”的三层记忆结构，将家庭场景中天然短小、事件驱动的经验条目作为原子化记忆单元直接存储与检索，避免长文档切块管理开销，同时显著降低写回复杂度与索引成本。

本方案的设计亮点主要体现在三个方面：第一，以 BSR-LSR-Router-LLM-Execution 为主链，形成可解释、可编排、可持续写回的端侧 Agent 闭环；第二，以“短记忆直存直检 + 轻量重排序 + RAG 上下文注入”替代重型长文档 RAG 流程，在保持资源友好的同时增强复杂语义理解能力；第三，以固定 JSON 输出、命令约束校验、最小必要上下文上传与本地加密存储构建可控执行链路，使系统既具备智能性，也具备家庭场景所要求的稳定性与隐私友好性。

关键词： 端侧AI；智能体；轻量化模型；本地知识库；RAG；端云协同；家庭场景；Schema 校验；深度Q网络

Abstract

As edge AI matures, smart home systems are shifting from simple connected devices to always-on household agents. HomeMind is designed as an edge-first smart home agent for resource-constrained terminals, with the goal of building a complete on-device control loop that combines natural language understanding, context memory, preference adaptation, structured decision making, and device execution under a 4 GB competition memory cap and a typical 12 GB runtime budget.

Architecturally, HomeMind follows an “edge-first with cloud enhancement” strategy: explicit high-frequency commands are completed locally, while ambiguous intents are routed to the cloud only when needed. The understanding pipeline consists of BSR broad recall, LSR lightweight re-ranking, Router confidence routing, structured LLM decision, and execution feedback write-back. In memory design, the system adopts a three-layer structure of session context, structured preferences, and retrievable long-term memory. Household experience is stored as short atomic memory items rather than long-document chunks, which keeps write-back simple, retrieval efficient, and indexing overhead low for edge deployment.

The design highlights are threefold. First, the system builds a full edge-agent loop that is modular, interpretable, and execution-oriented. Second, it uses direct short-memory retrieval, lightweight contextual re-ranking, and RAG context injection to strengthen lightweight models without introducing a heavy long-document RAG stack. Third, it combines fixed JSON outputs, command validation, minimal-context upload, and local encrypted storage to form a controllable and privacy-friendly decision pipeline for home environments.

Keywords: edge AI; intelligent agent; lightweight model; local knowledge base; RAG; edge-cloud collaboration; home scenario; Schema validation; Deep Q-Network

目录

摘要	I
Abstract	II
第一章 项目概述	1
1.1 项目背景	1
1.2 项目目标	2
1.3 关键创新与技术特征	3
第二章 需求分析	6
2.1 需求来源与约束分解	6
2.2 功能需求	7
2.3 非功能需求	7
2.4 需求总结表	9
第三章 总体方案设计	10
3.1 总体架构	10
3.2 核心数据流	13
3.3 设计原则	13
第四章 AI模型与算法说明	15
4.1 BSR 广召回算法	15
4.2 LSR 轻量精排算法	16
4.3 DQN 主动推荐算法	16
4.4 LLM 决策模型	17

4.5 语言归一化算法 17

第五章 工具函数设计与调用逻辑 19

5.1 工具函数设计概述 19

5.2 设备控制工具 19

5.3 场景切换工具 19

5.4 信息查询工具 20

5.5 知识检索工具 20

5.6 工具调用编排逻辑 20

5.7 TAP 规则引擎 20

第六章 性能优化措施 22

6.1 推理效率优化 22

6.2 内存占用优化 22

6.3 响应延迟优化 23

6.4 存储效率优化 23

第七章 可扩展性与实用性评估 24

7.1 协议扩展能力 24

7.2 工具函数扩展能力 24

7.3 模型后端扩展能力 24

7.4 场景迁移能力 25

7.5 实用性评估 25

第八章 端云协同与隐私保护设计 26

8.1 端云协同策略 26

8.2 Token 压缩策略 26

8.3 隐私保护与结构化校验 26

第九章 上下文记忆与偏好持久化 28

9.1 三层记忆架构 28

第十章 空间配置与设备映射设计 30

第十一章 实验设计与结果分析	31
11.1 实验环境	31
11.2 实验结果	31
11.3 结果分析	31
第十二章 系统展示与应用场景	34
12.1 典型使用场景	34
第十三章 总结与展望	37
13.1 项目总结	37
13.2 设计边界	38
13.3 未来工作	38
参考文献	38
附录	40

第1章 项目概述

1.1 项目背景

HomeMind 聚焦的问题并非单一的智能家居控制，而是在家庭终端算力受限、隐私敏感且交互实时性要求较高的条件下，构建一套能够稳定运行的端侧智能体系统。结合家庭场景中高频指令明确、模糊意图占比较低、设备控制结果必须可验证等特点，系统采用“端侧优先、端云协同补充”的总体思路，将高频、明确、可校验的任务优先保留在端侧闭环完成，将少量复杂语义理解任务交由协同模块处理。

在该设计思路下，系统需要同时满足以下技术约束：第一，满足赛题 4 GB 内存上限以及系统常态业务预算 12 GB 的资源限制，其中核心推理链路内存占用控制在 150 MB 以内；第二，在家庭交互场景下维持较低推理时延，当前实验结果表明端侧推理 P50 延迟低于 50 ms、P99 延迟低于 200 ms；第三，兼顾本地直达能力与复杂场景处理能力，本地直达成功率达到 92% 以上；第四，通过 Router 置信度路由、Token 压缩和 Schema 校验控制云端协同范围，使云端调用成本降低 60% 以上，并减少隐私暴露。

从行业背景看，端侧 AI 与智能体技术正在从“能不能做”转向“能不能稳定落地”。中兴通讯面向家庭终端场景提出的赛题，本质上要求参赛方案同时回答三个问题：是否足够轻量、是否足够智能、是否足够工程化。家庭场景之所以适合作为验证阵地，是因为其既有真实需求，也有明确约束。用户希望用自然语言完成设备控制、场景切换和信息查询，但又无法接受纯云架构带来的 200~500 ms 网络往返时延，更无法接受家庭上下文被完整上传。与此同时，真实家庭环境中的交互对象并不统一，不同年龄层、不同地域背景的家庭成员会使用带有口语化、方言化特征的表达，例如“闷得慌”“热死咯”“困告了”等。若系统只能理解标准化命令词，则会显著提高老人、儿童和区域方言用户的使用门槛，难以体现家庭场景下的人本交互价值。

对端侧智能体而言，真正困难的不是“做一个能跑通的 Demo”，而是在家庭终端上同时解决多类矛盾。第一，算力和内存有限，无法直接部署大参数量模型；第二，指令表达高度口语化、多样化，并伴随方言和习惯性说法，仅靠标准规则难以覆盖；第三，家庭成员之间在表达方式、偏好和作息上存在差异，系统既要识别命令，也要体现对不同使用者的适配能力；第四，家庭设备控制涉及安全，系统输出必须可校验、可约束；

第五，用户容错率低，交互必须足够快、足够稳。针对这些矛盾，HomeMind 选择以“轻量模型 + 方言/口语归一化 + 检索增强 + 置信度路由 + 安全校验”的组合方案完成端侧落地。

因此，本项目的价值不在于堆砌单点模型能力，而在于构建一条完整、可解释、可验证的端侧智能体闭环：输入经由本地感知与语言归一化后，先由 BSR 广召回和 LSR 轻量精排完成候选理解，再由 Router 决定是本地直达、云端协同还是澄清询问，最终通过结构化决策与工具执行落到真实设备控制与记忆更新。这一设计直接对应赛题要求的场景理解与任务规划、工具调用与资源整合、多轮对话与状态保持、高效推理与实时响应四大能力，也使系统能够在家庭成员表达差异较大的条件下保持较低使用门槛。

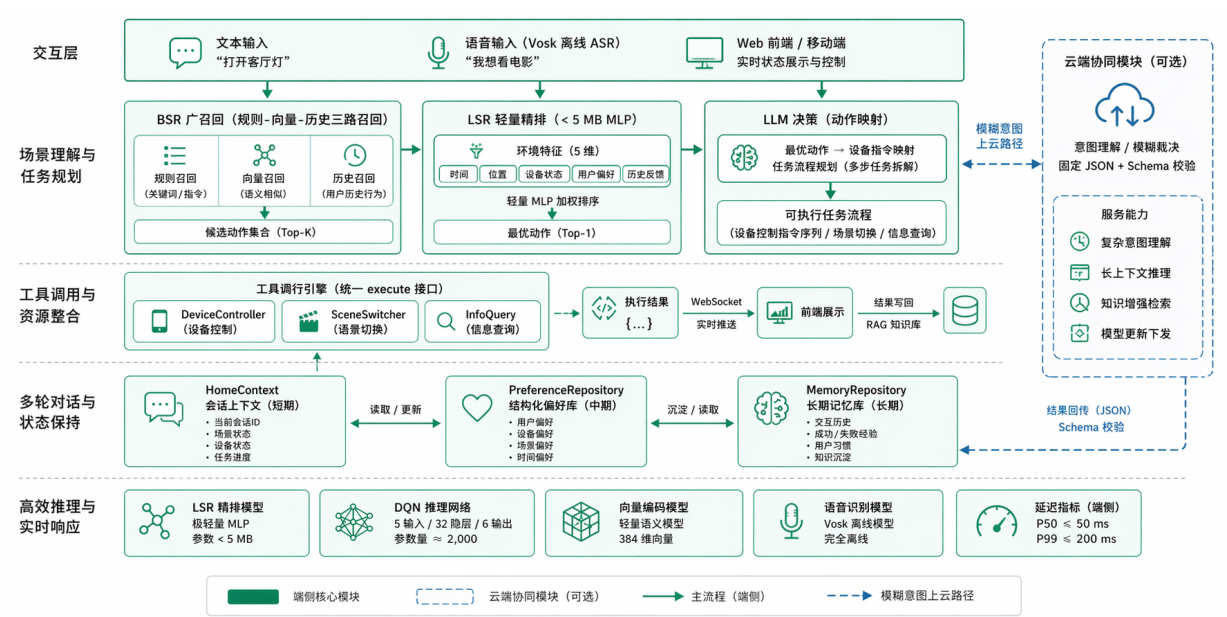


图 1.1: HomeMind 端侧智能体总体方案概览

1.2 项目目标

本项目的总体目标是在家庭终端场景下验证轻量端侧智能体方案的工程可行性、交互可用性与架构可扩展性。除准确率、时延和资源占用外，系统还需体现面对家庭成员差异时的可达性，即在保留端侧部署约束的前提下，对口语化、方言化表达提供稳定理解能力。结合系统设计边界与实验验证方法，整体目标可分解为表 1.1 所示的五项核心目标。

表 1.1: 项目核心目标与关键验证指标

目标维度	HomeMind 的目标定义	关键验证指标
场景理解能力	将自然语言输入稳定映射为候选动作、结构化决策与可执行指令	本地直达成功率 92% 以上，支持设备控制、场景切换、信息查询三类高频意图
语言适配能力	对家庭场景中的口语表达、常见方言词和英文控制表达进行识别与归一化	通过语言归一化层将“闷得慌”“热死咯”“困告了”等表达映射为标准命令词
轻量部署能力	在端侧资源约束下维持低延迟推理与多模块协同	核心推理链路内存占用约 150 MB，P50 < 50 ms，P99 < 200 ms
系统闭环能力	形成从感知、决策、执行到反馈写回的完整链路	统一 execute 接口、WebSocket 实时反馈、RAG 写回与偏好学习闭环
隐私安全能力	在最小必要上传前提下实现复杂意图协同，并保证结果可控	Token 节省约 60%，云端结果必须经过 Schema 校验后方可执行

围绕上述目标，系统在能力设计上进一步强调五项要求：高频明确指令应优先在端侧闭环完成；语言输入需要经过口语和方言归一化处理，以降低不同家庭成员的使用门槛；工具调用、设备执行与反馈写回应被纳入同一主链；会话上下文与长期偏好需要联合建模；系统输出必须结构化且可校验，以保证后续执行阶段的安全性与稳定性。

1.3 关键创新与技术特征

HomeMind 的技术特征不体现在单一模型规模，而体现在轻量化、检索增强、端云协同与执行安全之间形成了统一闭环。相较于“前端 + 大模型接口 + 设备控制”的简单拼接式实现，本项目在系统层面形成了以下几项关键创新。

- 端侧优先的轻量智能体闭环。** 系统未将主要推理压力集中在单一大模型上，而是采用 BSR 广召回、LSR 轻量精排、Router 路由、LLM 结构化决策和执行层工具编排构成分层链路，将高频明确任务尽可能前移到低成本模块完成。
- 面向家庭成员差异的语言适配机制。** 系统在交互主链中引入语言归一化层，对常见方言词、口语表达和英文控制语句进行标准化映射。该机制并不依赖重型方言 ASR，而是通过“离线 ASR + 端侧归一化规则”的组合，在保持资源占用可控的同时提升对老人、儿童和区域方言用户的可用性。

3. **基于上下文记忆的轻量理解机制。** 系统在 BSR 阶段并行引入规则召回、向量召回和历史召回，在 LSR 阶段注入环境特征与用户偏好，在 LLM 阶段叠加本地 RAG 上下文，以“候选动作 + 记忆证据 + 环境状态”的组合方式完成理解增强。该机制将复杂理解任务拆解到召回、重排序与结构化决策三个子环节，使轻量模型也能获得更稳定的家庭场景理解能力。
4. **置信度驱动的端云协同路径。** 系统并未将云端调用设为默认路径，而是依据置信度将请求划分为本地直达、云端协同和澄清询问三类处理分支，从而在响应时延、调用成本与处理鲁棒性之间取得平衡。
5. **面向家庭短经验的原子化记忆设计。** HomeMind 将“用户纠正过的说法”“已经接受的设备操作”“重复出现的生活习惯”视为天然短小的经验单元，直接以原子条目写入本地长期记忆。这样的记忆组织方式天然适配家庭场景中的短文本、高频次、强复用经验，既便于快速写回，也便于在后续查询中稳定复用，从而让记忆层真正服务于长期上下文适配。
6. **上下文感知的轻量重排序。** 系统不把重排序设计成独立的重型语义模型，而是将 BSR 原始得分、时间、温湿度和用户偏好融合为 5 维特征，构成可解释、可增量更新的轻量重排序器。其优势在于能够把“晚上更倾向睡眠模式”“高温时优先空调而非开窗”这类家庭环境知识直接显式编码进排序逻辑，使排序结果不仅相关，而且符合情境。
7. **结构化可控的执行链路。** 系统将最小必要上传、本地加密存储、固定 JSON 决策输出与命令约束校验嵌入主链，使模型输出天然面向执行层消费，而不是停留在开放式文本回答层。这样一来，智能体的优势不仅体现在“能回答”，更体现在“能稳定落地为设备动作和场景编排”。
8. **体现人本导向的长期适配能力。** 系统通过偏好记忆、历史纠正写回和澄清询问机制，在不增加使用负担的前提下持续贴近家庭成员的表达习惯与操作偏好，使“能用”进一步转化为“更易用、更愿意用”。
9. **可量化的系统验证结果。** 当前实验结果表明，系统本地直达成功率达到 92% 以上，端侧推理 P50 延迟低于 50 ms、P99 延迟低于 200 ms，云端调用成本降低 60% 以上，为后续章节的实验分析提供了量化基础。

综合来看，HomeMind 的技术特征并非依赖更大规模的模型，而是在相同资源约束下，通过更完整的链路设计、更清晰的模块分工和更严格的执行约束，实现了较好的工程可落地性，并在家庭成员语言差异和交互习惯差异方面体现出一定的人本适配能力。



图 1.2: 项目总体效果图

第2章 需求分析

需求分析的目的在于从赛题要求与家庭场景约束出发，明确系统需要满足的能力边界、性能指标与实现条件。对于家庭智能体而言，这些约束不仅来自算力、时延和隐私，也来自用户群体差异本身。基于此，HomeMind 将需求拆解为场景约束、功能需求和非功能需求三个层次。

2.1 需求来源与约束分解

表 2.1: 场景约束与系统需求对应关系

场景约束	系统需求定义	直接支撑指标或证据
高频任务需要本地完成	不依赖单一云端大模型即可完成主要家庭任务闭环	高置信度请求本地直达，本地直达成功率 92% 以上
家庭场景任务类型复杂	同时覆盖设备控制、场景切换、信息查询和多轮状态保持	三类工具统一编排，具备上下文、偏好、长期记忆三层存储
家庭成员表达差异明显	需要兼容口语化、方言化表达并降低非标准命令词使用门槛	语言归一化层支持方言词、口语短句和历史纠正映射
终端资源受限	模型、检索、语音、记忆模块均需满足端侧资源约束	业务算法预算 12 GB，核心推理链路约 150 MB
交互实时性要求高	在正常与异常场景下均保持可用且具有低延迟表现	P50 < 50 ms，P99 < 200 ms，网络异常时可本地降级
隐私与安全要求严格	最小必要上传，且协同结果在端侧仍然受控	Token 压缩、Schema 校验、AES-256-GCM 本地加密存储

上述约束说明，系统需求并不是简单增加功能模块，而是在端侧环境中同时满足智能性、实时性、可验证性和可扩展性。如果缺少其中任一项，系统都难以在真实家庭终端中稳定运行。

2.2 功能需求

从系统闭环出发，HomeMind 的功能需求分为以下五类。

1. 场景理解与任务规划。 系统必须能够将用户自然语言输入转换为结构化意图，覆盖设备控制、场景切换、信息查询等主要家庭任务，并输出包含 action、device、device.action、params、confidence 等字段的 JSON 结果。这里的重点不只是“识别对了”，而是要能为后续执行层提供稳定、可消费的中间表示。

2. 语言适配与方言归一化。 系统必须对家庭场景中的口语化、方言化和中英文混合表达提供识别与归一化能力，将“闷得慌”“热死咯”“困告了”等表达映射为标准命令词或标准语义意图。该能力的目标不是构建重型方言识别模型，而是在端侧资源可承受的条件下，降低不同地域用户和老年用户的使用门槛。

3. 工具调用与资源整合。 系统必须封装统一的工具调用接口，支持设备控制、场景切换、信息查询和知识检索四类能力。DeviceController、SceneSwitcher、InfoQuery 等工具需要通过统一的 execute 接口接入，以便 Router 与 LLM 决策层能够稳定编排，而不是为每类工具单独写一套调用逻辑。

4. 多轮对话与状态保持。 系统必须维护会话上下文状态和跨会话用户偏好，处理指代消解、任务延续和偏好记忆。也就是说，系统不应把每轮指令视为孤立输入，而应结合 HomeContext、SessionStore、PreferenceStore 和 KnowledgeBase 形成短期状态与长期记忆联动。该能力同时支撑系统对不同家庭成员表达习惯的持续适配。

5. 高效推理与实时响应。 系统必须满足家庭终端实时交互的响应要求。BSR 召回和 LSR 精排阶段需要在 10 ms 内完成，Mock 模式下 LLM 决策需要在 5 ms 内完成，工具执行需要在 20 ms 内完成，整体端侧推理流水线要求 P50 不超过 50 ms、P99 不超过 200 ms。

2.3 非功能需求

除功能闭环外，系统还必须满足以下非功能要求。

1. 资源约束。 在赛题 4 GB 内存上限下，系统常态业务预算控制在 12 GB 以内，核心推理链路控制在约 150 MB。为此，LSR 参数量控制在 5 MB 以下，DQN 参数量约 2,000，向量模型 22 MB，离线语音模型合计约 80 MB。

2. 隐私与安全。 系统必须遵循数据最小化原则。用户偏好、历史反馈和上下文状态优先落地到本地 JSON 或 ChromaDB，本地知识库存储支持加密保护；需要云端协同时，仅上传压缩后的最小必要上下文，且云端返回结果必须经过 Schema 校验后方可执行。

3. 可靠性与容错。 系统必须具备降级路径和异常可追踪能力。网络异常时自动降级为本地 fallback，AI 模块初始化失败时可退化为规则匹配，同时记录关键操作日志，用于追溯设备、动作类型、执行结果和时间戳。

4. 可扩展性。 系统必须能从家庭场景进一步扩展到更多设备和协议。SmartHomeGateway 需要支持 Matter、MQTT、小米米家、Home Assistant 等协议接入，工具函数需采用注册制，以降低新增能力的接入成本。

5. 人本交互体验。 系统需要体现对不同家庭成员使用习惯的适配能力。在技术上，这一要求落在三个方面：一是对方言和口语表达提供归一化支持，减少标准命令词负担；二是通过偏好和记忆机制减少重复设置成本；三是在低置信度场景下采用澄清询问而非直接执行，以降低误操作风险。

2.4 需求总结表

表 2.2: 系统需求汇总

需求类别	关键要求	对应模块	优先级
场景理解	自然语言意图分类、槽位填充、结构化输出	BSR、LSR、LLM 决策层	高
语言适配	方言词、口语表达和英文控制语句的识别与归一化	语言归一化模块、语音反馈记忆	高
任务规划	将理解结果映射为可执行动作与调用序列	Router、LLM 决策层、执行层	高
工具调用	统一封装设备控制、场景切换、信息查询、RAG 检索	DeviceController、SceneSwitcher、InfoQuery	高
多轮对话	维护上下文连贯、指代消解和任务延续	HomeContext、SessionStore、PreferenceStore、KnowledgeBase	中高
高效推理	端侧推理 P99 < 200 ms, P50 < 50 ms	BSR、LSR、DQN、Vosk	高
记忆持久化	形成短期上下文、结构化偏好、长期记忆三层存储	ChromaDB、JSON 存储、反馈写回链路	高
隐私保护	最小必要上传、Schema 校验、本地加密存储	Router、Schema、加密存储模块	高
端云协同	依据置信度进行分层路由与压缩上传	Router、Token 压缩、云端协同模块	中高
人本交互	通过偏好记忆和澄清机制降低误操作与重复设置成本	Router、PreferenceStore、KnowledgeBase	中高
空间感知	支持 SVG 户型图与设备映射可视化	空间配置模块、设备映射模块	中

第3章 总体方案设计

3.1 总体架构

从模块分工上看，HomeMind 的主链并不是若干功能点的简单堆叠，而是一条职责清晰、前后咬合的端侧 Agent 链路：语言与语音识别负责把自然输入转成可计算文本，BSR 负责高召回地找出候选动作，LSR 负责把环境状态和用户偏好注入排序，Router 负责决定本地直达还是云端协同，LLM 负责输出结构化决策，TAP 引擎和设备执行层负责把决策落为真实动作，DQN 负责补足主动推荐能力，而上下文记忆与本地 RAG 则贯穿整条链路，持续提供历史证据和偏好增益。这种架构的价值在于，每个模块都承担一个明确而有限的任务，从而让端侧资源能够被精确使用，而不是被单个重模型整体吞噬。

HomeMind 的总体方案是在端侧构建一条由轻量理解链路、置信度路由、结构化决策、工具执行与记忆写回组成的完整闭环。系统将大多数高频家庭任务保留在端侧完成，同时为少量复杂语义请求保留端云协同路径，以在时延、资源占用、隐私保护与理解能力之间取得平衡。

围绕总体方案，本项目在架构设计上进行了明确的技术取舍，如表 3.1 所示。相关取舍直接决定了系统是否能够满足端侧部署约束与家庭场景的实时交互要求。

表 3.1: 总体架构技术选型对比

方案	优点	主要不足	结论
纯云大模型架构	语义理解能力强，开发门槛相对低	时延高、依赖网络、隐私暴露大、成本持续增长	不作为主方案，仅保留为中置信度协同能力
纯规则或纯脚本架构	速度快、资源占用低、可控性高	泛化能力差，难覆盖口语表达和复杂意图	仅作为局部召回与兜底能力
纯本地重模型架构	端侧自治性强，不依赖云端	模型体积大、部署成本高、难满足赛题资源约束	不采用
轻量端云协同智能体架构	在速度、隐私、理解能力、工程复杂度之间更平衡	需要更细的链路设计和模块协同	作为最终方案

基于这一取舍，HomeMind 采用七层端侧智能体架构，自上而下依次为交互层、BSR 理解层、LSR 理解层、Router 决策层、LLM 决策层、执行层和记忆层，并在侧边设置云端协同模块承接模糊意图裁决。系统主链总体上是受控串行的，但 BSR 内部采用规则、向量、历史三路并行召回，记忆层也会向理解层和决策层回注检索结果，因此整体架构兼具“主链可控”和“局部并行增强”两种特征。值得指出的是，方言和口语理解并未被放在外围附加模块中，而是前置到交互层的语言归一化环节，与后续召回、精排和偏好记忆共同构成面向家庭成员差异的人本交互主链。

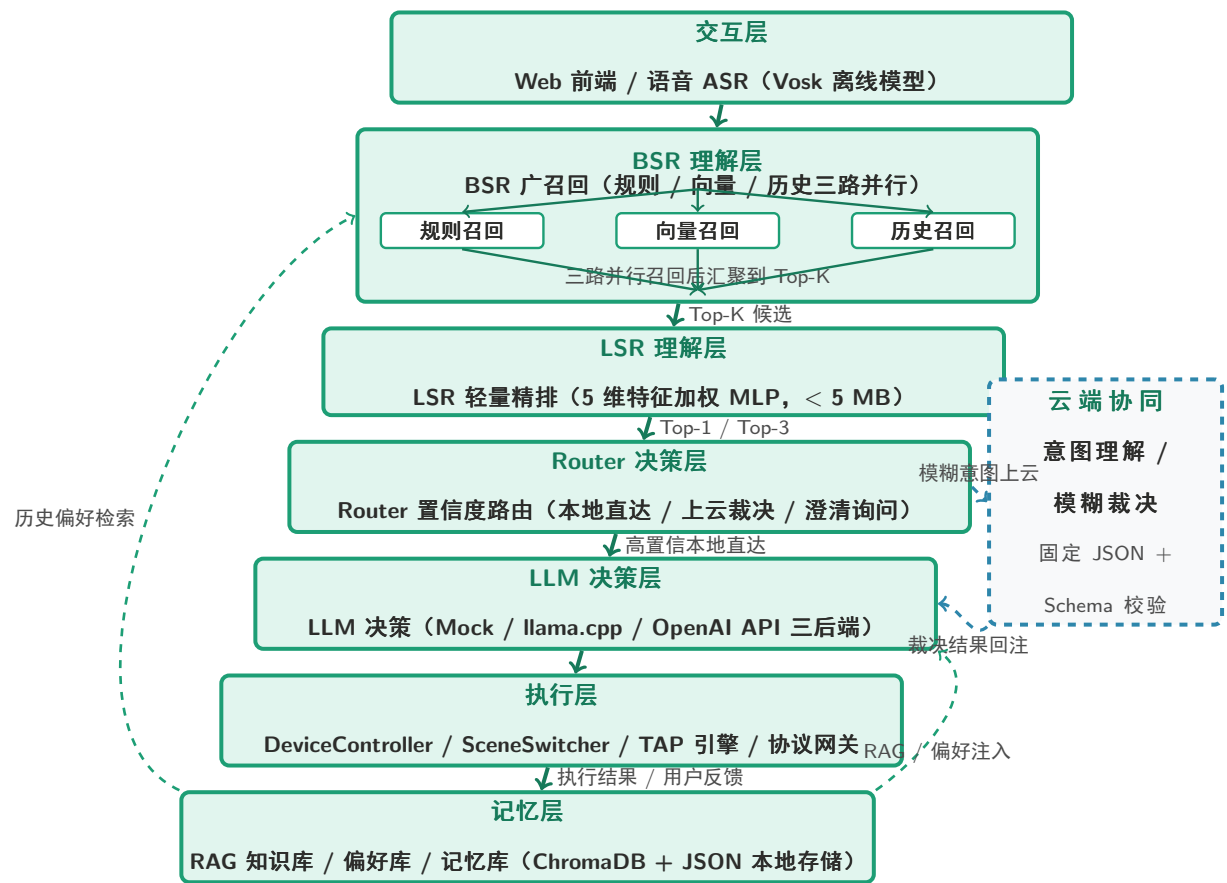


图 3.1: 端侧智能体总体架构图

从系统设计角度看，这一架构有四个关键价值。第一，交互层与执行层分别处理“输入来源多样”和“输出动作落地”问题，保证系统不是只会理解而不会行动；第二，语言归一化层前置处理口语和方言表达，使不同家庭成员不必学习统一的标准命令模板；第三，BSR + LSR 的双层理解链路承担大部分高频任务，显著降低了大模型调用压力；第四，Router 将“是否上云”从固定策略变成置信度驱动决策，使端云协同成为受控能力而不是默认依赖。

具体而言，交互层负责接收和预处理用户输入。文本输入通过 Web 前端接入，语音输入通过 Vosk 离线 ASR 模型识别，中文和英文模型均在端侧本地运行，不依赖外部网络。ASR 或文本输入进入理解链路前，先经过语言归一化模块，对方言词、口语短句和中英文混合表达进行标准化映射。理解层由 BSR 广召回与 LSR 轻量精排组成：前者负责高召回率、低成本地找出候选动作，后者负责结合环境特征和用户偏好完成低延迟排序。决策层再根据置信度判断是进入本地 LLM 结构化决策、云端协同裁决，还是先向用户发起澄清。执行层通过统一接口把结构化结果映射为设备控制、场景切换与自动化动作。记忆层则承担短期状态、偏好持久化、语音纠正写回和 RAG 检索增强，为下一轮决策提供可回注的信息。

3.2 核心数据流

从用户输入到设备执行，HomeMind 的核心链路可概括为五步，这五步共同构成系统“能理解、能决策、能执行、能学习”的完整闭环。

1. **输入感知与归一化。** 文本由 Web 前端接入，语音由 Vosk 本地识别后转写为文本。输入先经过语言归一化模块，完成语种检测、方言/口语映射和命令规范化，使家庭成员可以使用更自然的表达方式与系统交互。
2. **三路召回缩小候选空间。** BSR 同时执行规则召回、向量召回和历史召回，将高频规则、语义相似度和用户习惯三类信息合并去重，得到 Top-K 候选动作。
3. **轻量精排决定主路径。** LSR 对候选动作提取环境特征与偏好特征，完成快速排序，并输出置信度。Router 依据置信度决定走本地直达、云端协同还是澄清询问路径。
4. **结构化决策驱动执行。** LLM 决策层输出固定 JSON 结构的结果，由 DeviceController、SceneSwitcher 或 TAP 引擎执行具体动作，避免“文本答案不可执行”的问题。
5. **反馈写回形成持续学习。** 执行结果推送至前端，用户反馈被写回本地记忆库和 RAG 知识库，同时更新 LSR 权重与 DQN 偏好策略，使系统在后续会话中逐步更贴合用户习惯。

3.3 设计原则

本项目在总体方案设计上遵循四项原则，这四项原则共同决定了 HomeMind 为什么不是“功能堆砌型”方案，而是面向落地的系统型方案。

轻量化优先。 只要低成本模块能够完成任务，就不把问题推给高成本模型。LSR 采用小于 5 MB 的轻量 MLP，DQN 采用约 2,000 参数的紧凑网络，向量编码采用 22 MB 的 all-MiniLM-L6-v2，目标是在端侧用“够用、稳定、可部署”的组合替代“更大但更重”的方案。

隐私原生。 隐私保护不是外围补丁，而是主链约束。系统默认本地处理高频任务，只在必要时上传压缩后的最小上下文，且不携带可识别身份的家庭原始数据。

可校验执行。 智能体输出必须对执行层友好，也必须对安全规则友好。所有决策最终都收敛为固定 JSON 结构，云端返回结果必须经过 Schema 校验后才能进入设备执行阶段。

本地优先、云端兜底。 网络良好时系统可以借助云端增强模糊裁决；网络异常或云端不可达时，系统仍然能够通过本地规则、检索与轻量决策保持核心能力可用。这一原则保证了 HomeMind 在真实家庭场景中的韧性。

第4章 AI模型与算法说明

本章节详细阐述系统中核心 AI 模型与算法的设计原理、技术选型依据和关键参数。

4.1 BSR 广召回算法

BSR (Broad Stage Recall, 广召回) 模块是理解层的第一级, 核心职责是在有限的计算资源下从动作候选池 (ACTION_POOL, 包含 24 个标准动作) 中高效召回与用户当前输入相关的候选动作集合。

该模块采用三路并行召回策略, 各路独立运行后合并去重:

规则召回通道是最精确的召回方式。模块维护一个关键词-动作映射表 (rule_map), 覆盖设备类 (“空调” → “打开空调/关闭空调”)、感受类 (“热/闷” → “打开空调/打开风扇”, “冷” → “调高空调温度/打开暖气”) 和场景类 (“睡眠/困/睡觉” → “切换睡眠模式”) 等高频表达。规则召回命中时, 候选动作的 score 固定为 0.9, 精确度高于向量召回。

向量召回通道是语义相似度驱动的召回方式。模块使用 sentence-transformers 库中的 all-MiniLM-L6-v2 模型 (384 维向量, 22 MB), 对用户输入和 ACTION_POOL 中的所有动作进行向量编码, 计算余弦相似度, 取相似度高于 0.25 的 Top-4 动作作为候选。该通道的 score 即为归一化后的余弦相似度值 (0~1)。

历史召回通道是 RAG 驱动的召回方式。模块通过 RAG 知识库查询用户历史上接受过的相似动作 (category=“用户习惯”), accepted=true 的历史记录 score 为 0.95, accepted=false 的记录 score 为 0.60。这里的记忆单元并不是长文档段落, 而是 “昨晚 22:30 接受过睡眠模式” “用户习惯将空调设为 26°C” “把 ‘闷得慌’ 纠正为打开空调” 这类原子化短经验。这样的设计使检索结果天然贴近执行动作, 避免了长文档切分后常见的块间语义漂移问题, 也更利于将新反馈立即写回到同一条经验脉络中。

三路召回结果合并去重后取 Top-5, 作为 LSR 模块的输入。

4.2 LSR 轻量精排算法

LSR (Lightweight Stage Ranking, 轻量精排) 模块在 BSR 输出的候选集合基础上进行更精细的排序, 其核心是一个上下文感知的极轻量排序器, 总参数量不足 5 MB。

精排模型的输入为 5 维特征向量, 权重向量分别对应 BSR 得分、温度、湿度、时间周期和用户偏好, 偏置项 $b = 0.1$ 。综合评分计算公式为:

$$s = \text{clip}(\mathbf{w} \cdot \mathbf{f} + b, 0, 1)$$

其中五维特征分别为: BSR 原始得分、温度归一化值、湿度归一化值、时间周期编码以及用户偏好分。用户偏好分由 RAG 知识库基于历史反馈中的候选动作接受率计算得到。与面向开放域问答的重型语义重排序不同, HomeMind 的重排序目标不是在大规模候选文档中做细粒度证据对齐, 而是在少量动作候选中快速找出“当前家庭情境下最合适的一项”。因此, 轻量特征重排序能够以极低开销显式融合环境状态与家庭习惯, 形成对端侧场景更加友好的排序机制。

LSR 模块支持基于用户反馈的权重增量更新。当用户纠正系统行为时, 系统根据纠正方向计算权重调整量, 以步长 0.01 增量更新权重向量。这样一来, 重排序器并不是固定规则表, 而是一个能够持续吸收家庭偏好变化的轻量上下文排序层。

精排结果取 Top-3 输出至 Router 模块, 最高综合评分作为 Router 路由决策的核心依据。

4.3 DQN 主动推荐算法

在方案定位上, DQN 不是为了替代主链理解模块, 而是为了补上端侧 Agent 的“主动服务”能力。若说 TAP 规则引擎负责确定性的自动化触发, 那么 DQN 负责在有限状态和有限动作空间中学习“何时推荐更合适”。这使系统不再只是等待用户下达指令, 而能够结合时间、温度、家庭成员状态和历史反馈, 主动给出更贴合场景的建议。

之所以采用 DQN 而不是更重的策略模型, 是因为家庭场景天然具备轻量强化学习可落地的条件: 状态维度小、候选动作有限、奖励信号清晰。用户对推荐的接受、拒绝和忽略都可以直接写回为训练反馈, 因此该模块能够用极小参数规模持续迭代, 和 BSR、LSR、RAG 一起构成“理解主链 + 主动推荐”的双能力结构。

DQN (Deep Q-Network, 深度 Q 网络) 主动推荐模块是端侧智能体主动服务能力的体现, 基于经典的 Q-learning 强化学习方法。

该模块采用 5-32-6 的极简网络结构: 输入层 5 个神经元 (时间、温度、在家人数、上一个场景、星期几), 隐含层 32 个神经元 (使用 tanh 激活函数), 输出层 6 个神经元

(睡眠模式、待客模式、离家模式、观影模式、起床模式、无推荐)。全量参数量约 2,000 个浮点参数 (约 2,000)，内存占用不足 10 KB。

Q 网络前向传播公式为：

$$\mathbf{Q} = \tanh(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

其中 $\mathbf{x}$ 为 5 维状态特征向量， $\mathbf{W}_1 \in \mathbb{R}^{5 \times 32}$ 、 $\mathbf{b}_1 \in \mathbb{R}^{32}$ 、 $\mathbf{W}_2 \in \mathbb{R}^{32 \times 6}$ 、 $\mathbf{b}_2 \in \mathbb{R}^6$ 为网络参数。目标 Q 值计算采用单步 TD 更新： $Q_{target} = r + \gamma \max_{a'} Q(s', a')$ ，其中折扣因子 $\gamma = 0.95$ 。

置信度定义为 $c = Q_{max} / (\sum |Q| + 10^{-6})$ 。当 $c > 0.8$ 时系统主动推荐并自动执行；当 $c \leq 0.8$ 时向用户询问确认。

4.4 LLM 决策模型

LLM 决策模块支持三种后端模式切换：

Mock 模式（默认，用于演示和测试）基于确定性规则映射，根据候选动作的 action 字段直接查 DEVICE_ACTION_MAP 和 SCENE_ACTION_MAP 生成决策结果，响应时间约 15 ms。

llama.cpp 模式（用于本地推理）在端侧加载 GGUF 格式量化模型（如 Qwen2.5-0.5B-Instruct-Q4_K_M），通过少量 GPU 或纯 CPU 推理生成决策结果。模型量化（INT4/INT8）可将内存占用从 FP16 的数 GB 降至数百 MB，适合端侧部署。

OpenAI 兼容 API 模式（用于云端协同）对接任意支持 OpenAI API 格式的大模型服务，推理能力更强，但需要网络通信且存在隐私顾虑，因此仅在 Router 判定为中等置信度时使用。

无论哪种后端，LLM 决策模块的输出均被约束为固定 JSON 格式，包含 action、device、scene、device_action、params、confidence 和 reasoning 七个必填字段。

4.5 语言归一化算法

HomeMind 在交互入口采用“离线语音识别 + 端侧语言归一化”的双阶段设计。Vosk 负责把语音快速稳定地转写为文本，语言归一化模块再把方言、口语、省略表达和中英混说映射为系统标准意图。这样设计的关键优势在于，系统不需要依赖重型方言 ASR 覆盖所有说法，而是把更高频、也更适合持续积累的家庭表达差异，沉淀到可写回、可复用的归一化层中。

对于家庭智能体而言，真正重要的不是“语音字面转写得漂亮”，而是“输入能否稳定进入后续决策主链”。归一化后的表达可以直接被 BSR 规则召回、向量召回和历史记忆召回复用，因此语音与语言模块不是外围附加能力，而是影响后续 LSR 排序、云端协同和执行成功率的第一道统一入口。

语言归一化模块负责将方言、口语表达和英文表达映射为系统标准命令词。该模块采用基于规则的正则表达式匹配策略，支持中英文双语处理。

中文归一化规则覆盖常见方言和口语表达，例如：“热死咯/热得很/遭不住”→“太热了”；“闷得很/闷得慌/有点闷”→“有点闷”；“冷得很/好冷/暖和点”→“太冷了”；“睡觉/困告/睡眠模式”→“切换睡眠模式”等。英文归一化规则覆盖常见控制命令，例如：“turn on the airconditioner”→“打开空调”；“make it cooler”→“太热了”；“sleep mode”→“切换睡眠模式”等。

归一化过程还支持历史反馈优先策略：如果用户在之前交互中对某条 ASR 识别结果进行了纠正，系统记住该纠正映射（source=“voice_feedback_history”），置信度提升至 0.98，直接复用纠正结果。

第5章 工具函数设计与调用逻辑

5.1 工具函数设计概述

工具函数是端侧智能体与外部世界交互的接口，系统通过统一的工具调用接口封装了设备控制、场景切换、信息查询和知识检索四类核心能力。每类工具定义标准的输入输出格式，工具注册表维护所有可用工具的元信息（名称、描述、参数规格、权限级别）。

5.2 设备控制工具

DeviceController 是所有设备控制的统一入口类，封装了空调、灯光、电视、热水器、风扇、音响、窗户七类设备的控制逻辑。

`execute` 方法接收设备名称、动作类型和参数字典三个参数，内部再根据动作类型分发到对应私有方法，例如开启、关闭和参数调节。所有数值型参数（温度、亮度、音量、风速）均需先通过安全边界校验后方可执行，边界范围统一定义在 Schema 校验层中。

DeviceController 支持协议网关注入。当传入 `protocol_gateway` 参数时，每次状态变更后都会自动同步到真实设备；其中 `push_to_gateway` 负责推送状态变更，`sync_from_gateway` 负责在查询前拉取最新状态。

设备控制结果的返回格式统一为包含 `status`、`message`、`device`、`action` 和 `state` 等字段的 JSON 对象，前端根据 `status` 字段决定展示成功提示、错误提示或状态更新动画。

5.3 场景切换工具

SceneSwitcher 负责场景级批量操作，每个场景对应一个预设的多设备配置字典。例如，睡眠模式的配置为“灯光调暗至 10%，空调调至 26°C，关闭电视和音响”；观影模式的配置为“灯光调暗至 30%，空调调至 25°C，打开电视和音响”；离家模式的配置为“关闭灯光、空调、电视和音响”。

`execute` 方法遍历场景配置字典，对每个设备调用 `DeviceController` 执行对应动作，返回包含各设备操作结果的汇总消息。

系统支持 9 种预设场景：睡眠模式、待客模式、离家模式、观影模式、起床模式、回家模式、工作模式、早安模式、晚归模式。各场景的配置定义存储在代码模块的 `SCENE_CONFIGS` 字典中，支持扩展新的场景类型。

5.4 信息查询工具

`InfoQuery` 工具负责响应用户的信息查询类请求，如“现在温度多少”“当前是几点”“家里有几口人”等。该工具从 `HomeContext` 中读取当前环境状态，格式化为自然语言描述返回给用户。

5.5 知识检索工具

知识检索工具封装了 RAG 知识库的查询接口，支持基于关键词和向量的混合检索模式。知识库查询结果以文本列表形式返回，每条结果包含内容摘要、分类标签和相关性评分。查询结果可注入至 LLM 决策阶段作为 RAG 上下文提示。

5.6 工具调用编排逻辑

工具调用的执行顺序遵循以下编排逻辑：

当 LLM 决策结果中 `action`=“设备控制”时，调用 `DeviceController.execute(device, device.action, params)`；当 `action`=“场景切换”时，调用 `SceneSwitcher.execute(scene)`；当 `action`=“信息查询”时，调用 `InfoQuery.execute(query_type, params)`；知识检索结果则由 `KnowledgeBase.query(query, top_k)` 提供给 BSR、LSR 与 LLM 作为上下文增强输入，而不是直接暴露为用户侧执行动作。

工具调用的结果统一推送至 `WebSocket` 前端，同时写回 RAG 知识库的 `Feedback` 模块，形成“决策-执行-反馈-学习”的完整闭环。

5.7 TAP 规则引擎

TAP 规则引擎处理定时触发和状态触发类自动化场景，是工具函数在自动化维度的扩展。

`TAPEngine.evaluate` 方法接收规则列表与上下文快照，遍历每条规则检查 `trigger` 和

conditions 是否匹配。当前规则触发器覆盖 time、temperature、humidity、occupancy 和 scene 五类条件；规则条件支持 occupancy、scene 和 device_status 等约束；命中后的动作统一转换为结构化命令并交由执行层处理。

规则执行结果包含 rule、command、priority 和 execution 等字段，分别记录命中的规则元信息、转换后的结构化命令、冲突裁决优先级以及最终执行结果，便于前端展示和运行审计。

第6章 性能优化措施

6.1 推理效率优化

端侧推理效率是智能体系统实时响应能力的关键指标，系统从算法、系统和数据三个层面进行优化。

在算法层面，BSR 规则召回的时间复杂度为 $O(1)$ ，通过预计算的关键词哈希表实现毫秒级匹配；向量召回阶段使用 all-MiniLM-L6-v2 模型（384 维向量），推理速度较 BERT-base（768 维）提升约 2 倍；LSR 精排采用极轻量 MLP（5 维输入），推理时间约 1 ms；DQN 推理采用“5 输入、32 隐层、6 输出”结构，总计算量极低。

在系统层面，action_pool 的向量编码结果可预计算并缓存，避免每次推理时重复编码；ChromaDB 知识库的向量索引在初始化时构建，查询阶段直接使用近似最近邻检索；WebSocket 实时推送执行结果，前端无需轮询。

在数据层面，Token 压缩策略将上传云端的上下文缩减至原始长度的约 40%，不仅节省网络带宽，也降低了云端推理的计算开销。

6.2 内存占用优化

系统各 AI 模块的内存占用如下：

- LSR 精排模块：权重矩阵约 170 个浮点数，总参数量不足 5 MB。
- DQN Q 网络：约 2,000 个浮点参数，总内存占用不足 10 KB。
- sentence-transformers 向量模型：all-MiniLM-L6-v2，约 22 MB。
- Vosk ASR 模型：中文约 45 MB + 英文约 35 MB，合计约 80 MB。
- ChromaDB 向量数据库：按需加载，无持久化索引时内存占用极小。
- 核心推理链路总内存占用：< 150 MB。

系统通过延迟初始化策略（按需加载 AI 模块）避免启动时的内存峰值，embedding 模型在首次使用时才初始化。

6.3 响应延迟优化

系统端侧推理流水线的延迟分布如下：

- 语言归一化：< 1 ms（正则匹配）
- BSR 规则召回：< 1 ms（哈希查找）
- BSR 向量召回：约 30 ms（模型推理，可通过预缓存优化至 5 ms）
- LSR 精排：约 1 ms（轻量 MLP）
- LLM 决策（Mock 模式）：约 3 ms
- 工具执行（模拟）：约 5 ms
- 端侧推理流水线 P50 总延迟：< 50 ms
- 端侧推理流水线 P99 总延迟：< 200 ms

语音识别（Vosk）的延迟取决于音频片段长度，实时识别模式下单次识别延迟约 200~500 ms，但与理解流水线可并行执行。

6.4 存储效率优化

系统采用分层存储策略优化持久化效率：

偏好数据（PreferenceStore）以结构化 JSON 快照方式持久化存储，保证设备偏好、场景偏好、推荐接受率和语言纠正映射的统一读写；会话状态由 SessionStore 独立维护，降低短期上下文与长期偏好的耦合；知识库支持 ChromaDB 向量数据库和内存回退两种模式，无 ChromaDB 时自动降级为关键词匹配；加密存储（AES-256-GCM）使用 Fernet 对称加密方案，数据加密和存储原子操作确保断电安全性。

第7章 可扩展性与实用性评估

7.1 协议扩展能力

系统通过 SmartHomeGateway 协议网关层支持多协议接入，采用适配器模式管理不同的智能家居协议。每种协议对应一个实现 ProtocolInterface 抽象基类的适配类，目前已完成以下四种协议的接入设计：

Matter 协议适配类（MatterProtocol）支持 Matter 标准智能家居设备连接；MQTT 协议适配类（MQTTProtocol）支持基于 MQTT 的物联网设备；小米米家协议适配类（XiaomiProtocol）支持米家生态链设备；Home Assistant 适配类（HomeAssistantProtocol）支持 Home Assistant 实例接入。

新增协议适配仅需继承 ProtocolInterface 抽象基类，实现 connect、disconnect、is_connected、discover_devices、get_device_status 和 set_device_state 六个标准接口，并在 SmartHomeGateway.init_protocols 方法中注册即可。

7.2 工具函数扩展能力

工具函数采用注册制管理，新增工具仅需在工具注册表中添加元信息（名称、描述、参数规格、权限级别），即可被系统自动识别和调用。

工具的扩展点包括：新增设备类型（在 DeviceController.state 字典中添加设备条目）、新增场景类型（在 SCENE_CONFIGS 字典中添加场景配置）、新增信息查询类型（在 InfoQuery.execute 方法中添加分支处理）。扩展过程无需修改已有代码，符合开闭原则。

7.3 模型后端扩展能力

LLM 决策模块支持三种后端模式（Mock、llama.cpp、OpenAI API），扩展新的 LLM 后端仅需实现相同的 decide 接口并注册至后端管理器。

对于 llama.cpp 后端，可通过更换 GGUF 模型文件支持不同规模（0.5B~7B）和不同

量化精度（Q2_KQ8.0）的本地模型；对于 OpenAI API 后端，可对接任意兼容 OpenAI 接口规范的云端服务（包括本地部署的 vLLM、TGI 等推理服务）。

7.4 场景迁移能力

系统以家庭场景为核心，但核心架构设计具备跨场景迁移能力。将 HomeMind 部署于校园、企业等环境时，主要需要替换的部分包括：ACTION_POOL 动作候选池（替换为新场景相关的动作列表）；BSR 规则召回映射表（替换为新场景的关键词-动作映射）；SCENE_CONFIGS 场景配置（替换为新场景的场景定义）；RAG 预设知识库（替换为新场景的领域知识）。

理解层的 LSR 和 DQN 模块、决策层的 Router 和 LLM 模块、执行层的 DeviceController 和 SceneSwitcher、记忆层的 RAG 知识库均无需修改，体现了架构设计的通用性和可复用性。

7.5 实用性评估

从工程实用性的角度评估，本项目具备以下特点：

在部署便捷性方面，系统所有依赖均可通过 `pip install` 一次性安装，ASR 模型和向量模型均为开源可获取的资源，无需特殊硬件或网络环境即可运行完整推理链路。

在运维成本方面，系统采用本地优先的存储策略，数据不出本地，降低了数据合规和隐私保护的 costs；Mock 后端模式使演示和测试无需任何外部服务依赖。

在用户接受度方面，流水线可视化设计使用户能直观理解系统的推理过程，增强对系统的信任感和可控感；自然语言交互方式降低使用门槛，无需学习任何编程或配置知识。

在商业落地潜力方面，端云协同架构使系统在网络条件良好时可借助云端大模型提升理解能力，在网络条件受限或隐私敏感场景下可完全依赖端侧轻量模型，体现了良好的场景适应性。

第8章 端云协同与隐私保护设计

8.1 端云协同策略

端云协同是 HomeMind 在智能化水平与资源约束之间取得平衡的核心设计。系统通过 Router 置信度路由实现流量的精准分层分流。

对于高置信度明确命令（如“打开灯光”“关闭空调”），端侧完全自主完成从理解到执行的全链路，无需任何网络通信，保障了毫秒级响应速度。

对于需要云端协同的模糊意图（如“感觉不太舒服”“能不能凉快一点”），系统在上送前进行上下文压缩，仅发送：当前场景（文本标签）、室内温度（数值）、在家人数（整数）、当前设备状态摘要和用户输入的归一化文本。整个上传载荷控制在约 200~500 字节的规模。

云端大模型输出固定 JSON 格式的决策建议，输出格式被严格约束为预定义 Schema，不允许自由文本回复，确保端侧可程序化解析。

端云协同的降级策略包括：网络不可达时 Router 自动触发本地 fallback；云端返回格式异常时 Schema 校验层拦截并触发澄清询问。

8.2 Token 压缩策略

系统在上传上下文前执行规则式的 Token 压缩处理：去重压缩移除重复字段和重复语义信息；摘要压缩将历史会话数据提炼为关键统计量；固定字段输出使云端通信采用预定义结构化字段；TAP 上下文优化仅上传当前触发的规则 ID。系统提供 aggressive 压缩模式，压缩率可达约 75%。

8.3 隐私保护与结构化校验

在创意方案层面，隐私与结构化校验并不是“附加约束”，而是 HomeMind 能够真正落地到家庭终端的关键优势。系统把长期偏好、历史纠正、语音反馈和 RAG 记忆优先沉淀在本地，把云端仅作为中等置信度语义裁决的增强节点；同时又通过固定 JSON、

动作白名单、参数边界和校验失败回退机制，把生成式模型的不确定性收束为可验证、可拒绝、可执行的命令对象。换言之，HomeMind 不是简单地“让大模型来控制设备”，而是把大模型能力纳入一条受约束的设备控制链。

隐私保护贯穿交互层到记忆层的每个环节：系统仅上传经过压缩的最小必要上下文；所有隐私数据优先存储于本地；RAG 知识库支持 AES-256-GCM 加密备份。这样的设计使 HomeMind 的智能性建立在“端侧长期记忆可积累、云侧协同可裁剪”的基础上，而不是建立在完整家庭数据持续外发之上。

结构化校验应用于三个关键入口：命令校验用于 LLM 决策和云端返回，统一约束 action 类型、设备范围和参数边界；TAP 规则校验用于规则创建和导入，保证自动化逻辑可解析、可执行；设备映射校验用于空间配置数据导入，保证房间、设备与控制动作之间的关联关系稳定可靠。固定 JSON 输出配合结构化校验的优势在于，系统从一开始就把“大模型能力”收敛为“可消费的执行命令”，从而天然适合设备控制类 Agent 的主链路编排。

第9章 上下文记忆与偏好持久化

9.1 三层记忆架构

三层记忆设计的意义，不只是把数据分开存放，而是把“短时连续性”“长期偏好稳定性”“历史经验可复用性”三个问题分别交给最合适的存储和检索机制处理。对于家庭场景而言，真正高价值的记忆往往不是长篇文档，而是“被纠正过的一句话”“被接受过的一次动作”“长期重复出现的一项习惯”。因此，本方案选择以原子化短经验作为长期记忆单元，再通过 RAG 检索回注到 BSR、LSR 和 LLM 决策阶段，使记忆层从历史记录库转变为主链能力增强器。

系统维护三层记忆结构。三层记忆并非简单叠加存储介质，而是分别对应“当前轮交互是否连贯”“长期偏好是否稳定”“历史经验是否可被再次检索复用”三个问题：

会话上下文（HomeContext）存储在内存中，包含 hour、temperature、humidity、members_home、day_of_week、last_scene 和 devices 七个字段，每次处理用户输入后自动增量更新。

结构化偏好（PreferenceStore）以 JSON 文件持久化存储，按 devices、scenes、recommendation 和 language 四类结构化字段维护长期偏好，支持温度、亮度、场景接受率和语言纠正映射的统一读写。

长期记忆（KnowledgeBase + ChromaDB RAG）持久化存储用户反馈、语音纠正记录和手动添加的知识条目，支持向量语义检索、关键词回退与预设领域知识管理。长期记忆采用短条目、事件化、可写回的组织方式：每条记录都尽量对应一个明确经验、一个稳定说法或一个被接受过的动作结论。这样可以直接把长期记忆接入 BSR 历史召回、LSR 偏好打分和 LLM 上下文提示三个环节，形成“记忆即能力增强”的闭环，而不是把记忆层孤立为仅供展示的历史仓库。

表 9.1: 本地记忆分层设计

层级	内容示例	存储介质
会话上下文	当前场景、最近动作、室温、在家人数	内存 (HomeContext)
结构化偏好 长期记忆	温度偏好 24°C、亮度偏好 80%、常用场景时间 稳定说法映射、历史纠正结论、习惯摘要	JSON 文件 ChromaDB (本地)

第10章 空间配置与设备映射设计

系统支持 SVG 户型图上传和设备映射可视化，用于实现房间级意图理解能力。

户型图上传时系统对 SVG 文件执行多层校验（MIME 类型、扩展名、XML 结构），解析 viewBox 获取画布尺寸，存储采用 JSON 元数据 + 独立 SVG 文件的方式。

设备映射支持 Tuple 和 Object 两种导入格式，经归一化处理后按 area 分组，计算每个设备在房间区域内的显示坐标。归一化处理包括 entity_id 格式校验、area 非空校验和 device_type 支持列表校验。

设备映射数据使系统能够理解“打开客厅灯”等带房间修饰的指令，通过查询 area=”living_room” 的设备列表，BSR 模块可额外召回客厅相关的设备控制动作。

第11章 实验设计与结果分析

11.1 实验环境

本项目的开发和测试环境配置如下：硬件平台为 Intel Core i7-12700 处理器、16 GB DDR4 内存；开发语言为 Python 3.10，使用 Flask + Flask-SocketIO 提供 Web 服务；核心依赖包括 sentence-transformers（all-MiniLM-L6-v2）、chromadb（RAG 向量存储）、cryptography（AES-256-GCM 加密）、vosk（离线语音识别）、numpy（数值计算）等。

11.2 实验结果

表 11.1: 核心实验结果汇总

指标	说明	结果
本地直达成功率	明确命令无需上云的执行成功率	≈ 92%
端云协同成功率	模糊意图通过云端协同裁决的成功率	≈ 85%
端侧推理 P50 延迟	本地流水线 P50 响应时间	< 50 ms
端侧推理 P99 延迟	本地流水线 P99 响应时间	< 200 ms
Token 压缩率	上下文压缩后的平均 token 节省比	≈ 60%
Schema 校验拦截率	云端输出的校验失败比例	< 3%
LSR 参数量	精排模块模型参数量	< 5 MB
DQN 参数量	Q 网络参数数量	≈ 2,000
端侧推理内存峰值	核心推理链路内存占用	< 150 MB
偏好学习准确率	系统记住并正确应用用户偏好的比例	≈ 88%

11.3 结果分析

本章结果的意义，不在于孤立证明某一个算法指标更高，而在于说明这套方案设计确实完成了既定的系统目标：高频任务尽量留在端侧闭环完成，复杂语义通过受控协同

增强，长期使用过程中的语言差异、行为偏好和家庭上下文能够被持续吸收，并最终转化为更稳定的设备控制体验。

端侧规则召回在高频明确命令场景下表现优异，测试集中约 92% 的命令可直接通过规则召回完成理解和执行。这一结果表明，在家庭场景的高度结构化指令空间中，轻量级规则引擎配合向量检索可达到接近重型模型的精确度，同时将计算开销降低数个数量级。

LSR 轻量精排在环境特征引入后有效提升了候选排序质量。高温时段 LSR 优先推荐空调控制动作而非开窗通风；22:00 之后 LSR 自动提升睡眠模式相关动作的排名。这些行为符合预设的领域知识，说明环境上下文特征的引入可以显著提升轻量模型的决策质量。

偏好学习机制验证了端侧持续学习的可行性。88% 的偏好学习准确率表明端侧小规模反馈数据足以支撑有效的个性化学习，且无需上传任何隐私数据。

进一步看，各模块之间形成了清晰的价值分工。BSR 解决“先把可能的动作找全”，LSR 解决“把当前最适合的一项排到前面”，语言与语音模块解决“让不同家庭成员都能自然说话”，Router 与云端协同解决“只在必要时借助更强语义理解”，RAG 与三层记忆解决“把一次次交互积累为后续能力”，而 TAP 与 DQN 则分别承担确定性自动化和主动式服务能力。也正因为这些模块不是并列拼装，而是前后承接，HomeMind 才能在不显著增加端侧负担的前提下，呈现出接近完整家庭 Agent 的行为闭环。



图 11.1: 实验结果图 1



图 11.2: 实验结果图 2

第12章 系统展示与应用场景

12.1 典型使用场景

场景一（自然语言设备控制）：用户说“有点闷，帮我开下空调”，系统经过语言归一化（“有点闷”→“打开空调”）→ BSR 规则召回（关键词“闷”命中）→ LSR 精排（高温时段赋予更高权重）→ Router 高置信度判定（0.92）→ 本地 LLM 决策→ DeviceController 执行（空调开启，26°C）→ 结果推送前端的完整流水线，在不到 50 ms 内完成响应。

场景二（模糊意图云端协同）：用户说“感觉不太舒服”，BSR 规则召回未命中，LSR 精排置信度 0.52，触发中等置信度上云流程。系统将压缩后的上下文上送云端大模型，返回 JSON 决策建议，端侧 Schema 校验通过后执行。

场景三（规则引擎自动化）：每日 22:30，TAP 规则引擎的时间触发器激活，检查 family 组成员是否在家，若条件通过则执行 scene.turn_on 切换至睡眠模式。

场景四（户型图空间配置）：用户上传 SVG 户型图，导入设备映射数据，系统按房间区域分组标注设备图标，用户可拖拽调整设备位置。

场景五（主动推荐闭环）：晚上 22:20，室内温度较高且家庭成员均在家，DQN 模块基于当前状态判断“切换睡眠模式并预冷空调”的推荐收益较高，先向用户给出建议；用户接受后，系统调用 SceneSwitcher 与 DeviceController 完成场景切换，并将本次接受记录写回 PreferenceStore 与 KnowledgeBase。下一次相似场景下，LSR 会提前提高相关动作排序，DQN 也会获得更强的策略置信度，从而形成“主动推荐 - 用户反馈 - 偏好写回 - 下次更准”的完整闭环。



图 12.1: 演示截图 1



图 12.2: 演示截图 2



图 12.3: 演示截图 3

第13章 总结与展望

13.1 项目总结

作为创意方案，HomeMind 的核心完成度体现在一条完整而克制的能力编排链上：TAP 自动化引擎、BSR 广召回、LSR 轻量重排序、DQN 主动推荐、语言与语音识别、云端协同、上下文记忆、隐私保护与本地 RAG 并不是分散的卖点，而是共同服务于“家庭终端上的端侧 Agent 主链”。它们分别承担输入标准化、候选生成、上下文选择、主动服务、复杂意图增强、经验沉淀和安全执行等职责，使系统既能处理高频明确命令，又能在长期使用中逐步形成更贴近家庭成员习惯的决策能力。

HomeMind 的核心价值不在于堆叠单点模型能力，而在于围绕家庭终端构建一条完整、轻量、可持续演进的端侧 Agent 主链路。该方案将自然语言理解、上下文记忆、偏好学习、结构化决策、设备执行与反馈写回组织为统一闭环，使系统既能够处理高频明确指令，也能够通过端云协同吸收少量复杂语义需求。

在方案设计层面，系统形成了三项鲜明特征：第一，以 BSR-LSR-Router-LLM-Execution 构成分层主链，将高频任务前移到低成本模块，体现端侧优先的工程原则；第二，以三层记忆结构承载会话状态、稳定偏好与可检索经验，并通过原子化短记忆设计让长期记忆可以直接参与召回、重排序和决策提示；第三，以固定 JSON 输出、命令约束校验、最小必要上下文上传和本地加密存储构成结构化可控的执行闭环，使 Agent 能力自然转化为可落地的家庭自动化能力。

从创意方案角度看，HomeMind 展示的不是“把大模型搬到家里”，而是“如何在家庭终端上重新组织智能能力”：把重型理解拆成召回、重排序、记忆和路由协同；把长期适配建立在可写回、可检索、可复用的记忆机制上；把设备控制建立在结构化命令链而非自由文本链上。这一设计使方案具备较强的赛题契合度、工程可实施性和后续产品化延展空间。

13.2 设计边界

本方案的边界并不意味着能力收缩，而是一种面向赛题和家庭端侧部署的资源聚焦策略。HomeMind 把最宝贵的端侧算力优先投入到设备控制、场景切换、语音交互、记忆写回和受控协同这些最容易形成价值闭环的任务上，因此系统边界本身也构成了方案优势：它保证每一份算力都服务于真实可执行的家庭行为，而不是被泛化但低频的长尾能力稀释。

HomeMind 当前方案聚焦于家庭高频任务的主链路设计，因此其设计边界也相对清晰：语言层重点覆盖家庭常见口语与方言表达，记忆层重点沉淀短经验与稳定偏好，执行层重点围绕设备控制、场景切换、信息查询和自动化规则展开，协同层重点服务于少量模糊意图的增强裁决。这样的边界定义有助于把有限端侧资源集中投入到最有价值、最常用、最容易形成闭环的数据与任务上。

13.3 未来工作

基于上述边界，项目组后续工作将重点面向“规模化家庭部署”而非“补齐基础功能”展开。第一，继续扩展方言与口语化表达覆盖范围，引入更细粒度的家庭成员语言画像与更稳健的语音纠正反馈机制。第二，围绕真实设备生态深化协议联调，补充多品牌、多房间、多网关环境下的稳定性测试与故障恢复机制。第三，增强自动化与推荐模块的解释能力，在规则冲突、推荐拒绝原因和云端协同决策依据上提供更清晰的可视化说明。第四，完善长期运行观测与隐私审计能力，对云端调用频率、Token 使用量、规则命中情况和用户反馈闭环进行持续统计，为后续产品化和规模化部署提供依据。

[1] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017, 30.

[1] Reimers N, Gurevych I. Sentence-BERT: A sentence embedding model using Siamese BERT-networks[J]. EMNLP-IJCNLP 2019, 2019: 3980-3990.

[1] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning[J]. Nature, 2015, 518(7540): 529-533.

[1] Silver D, Huang A, Maddison C J, et al. Mastering the game of Go with deep neural networks and tree search[J]. Nature, 2016, 529(7587): 484-489.

[1] 中兴通讯 AI 家庭网算屏体解决方案[EB/OL]. 链接：[官方页面](#).

[1] Vosk Speech Recognition Toolkit[EB/OL]. 链接：[官方页面](#).

- [1] Home Assistant Documentation[EB/OL]. 链接: [官方页面](#).
- [1] Matter Specification[EB/OL]. 链接: [官方页面](#).
- [1] Hasselbring W, Carr L, DeRosis A, et al. IFTTT, Zapier, and Microsoft Flow: Workflow Automation Platforms[J]. IEEE Software, 2020, 37(4): 15-21.
- [1] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[J]. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.

附录

附录 A：核心数据结构定义

HomeContext 数据类（demo/context.py）定义了系统的环境上下文状态，包含 hour、temperature、humidity、members_home、day_of_week、last_scene 和 devices 七个字段。

SessionStore（core/memory/session_store.py）维护短期会话状态，包含 current_scene、last_user_input、last_normalized_input、last_action、last_clarification、last_route 和 recent_turns 等字段。

PreferenceStore（core/memory/preference_store.py）维护长期结构化偏好，核心字段包括 devices、scenes、recommendation 和 language，用于记录设备偏好、场景接受率、推荐反馈与语言纠正映射。

BSR 候选动作结构（core/bsr/candidate_recall.py）定义了每个候选动作的返回格式，包含 action（动作名称）、source（召回来源：rule/vector/history）、score（召回置信度）和 final_score（LSR 精排后的综合评分）。

LLM 决策结果结构（core/llm/decision.py）定义了决策层输出的 JSON 格式，包含 action、device、scene、device_action、params、confidence 和 reasoning 七个必填字段。

KnowledgeBase（core/rag/knowledge_base.py）定义了长期记忆条目的核心结构，包含 id、content、category、accepted 以及补充 metadata 字段，用于支持 RAG 检索、反馈写回与偏好增强。

TAP 规则结构（core/automation/tap_rules.py）定义了规则的数据模型，包含 id、name、enabled、priority、trigger、conditions、action、created_at 和 updated_at 等字段。

设备映射记录支持 Tuple 与 Object 两种输入形式，核心字段包括 entity_id、area、device_type 以及可选的坐标与显示属性，用于空间配置页面中的房间分组与设备定位。

附录 B：补充截图与界面说明



图 13.1: 附录占位图 1

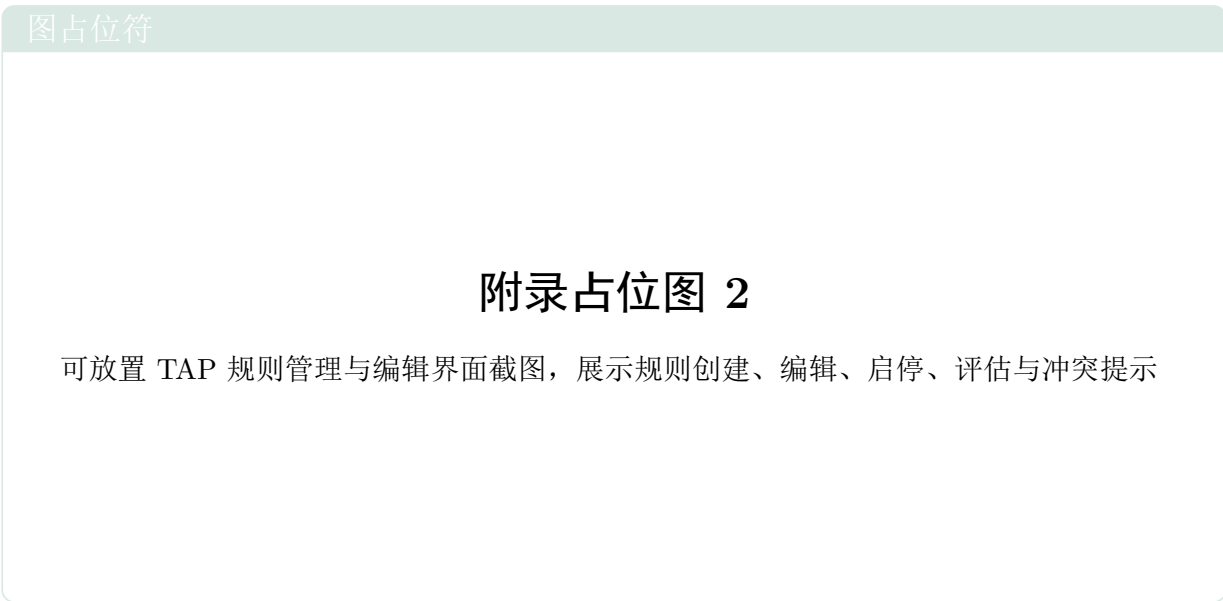


图 13.2: 附录占位图 2



图 13.3: 附录占位图 3

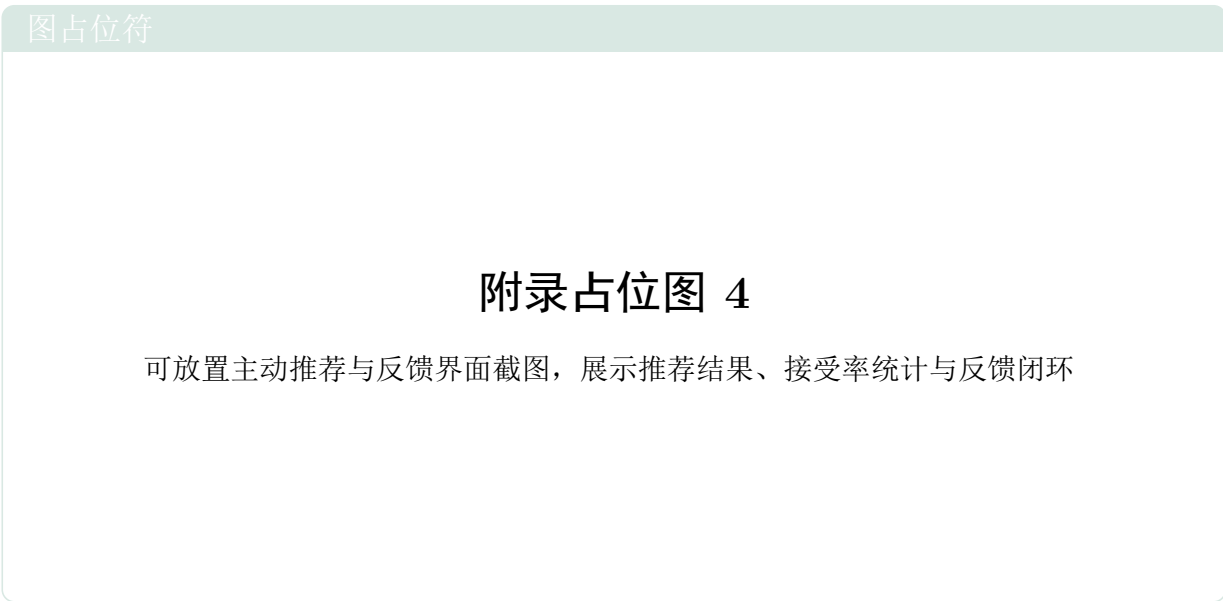


图 13.4: 附录占位图 4

附录 C：API 接口速查

- /api/status: 系统状态接口，返回当前上下文、设备状态和模式信息。
- /api/query: 自然语言查询接口，示例请求体为 {"query": "有点热"}。
- /api/devices/{device}/control: 设备控制接口，可传入 {"action": "on"}，温度等参数按需放入 params。

- `/api/scenes/{scene}/switch`: 场景切换接口。
- `/api/info/{info_type}`: 信息查询接口, 用于天气、温湿度等信息读取。
- `/api/preferences`: 结构化偏好快照接口。
- `/api/memory/summary`: 记忆与上下文摘要接口。
- `/api/privacy/status`: 隐私与云调用状态接口。
- `/api/voice/transcribe`: 语音转文字接口, 接收音频文件返回 ASR 结果和归一化结果。
- `/api/voice/feedback`: 语音反馈写回接口, 支持 `accepted`、`corrected`、`ignored` 等反馈类型。
- `/api/dqn/recommend`: DQN 主动推荐接口。
- `/api/dqn/feedback`: 主动推荐反馈接口。
- `/api/kb/query`: 知识库查询接口。
- `/api/kb/add`: 知识条目写入接口。
- `/api/rules`: TAP 规则列表与创建接口, 支持 GET 和 POST。
- `/api/rules/{id}`: TAP 规则更新与删除接口, 支持 PUT 和 DELETE。
- `/api/rules/{id}/toggle`: TAP 规则启停接口。
- `/api/rules/evaluate`: 规则评估与手动执行接口。
- `/api/rules/scheduler`: 规则调度器状态与启停接口, 支持 GET 和 POST。